EXP11

Septemeber 23, 2026

1 Multilayer Feed-Forward Network (MLP) on Wine Quality Dataset

[ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report,␣
↪confusion_matrix
```

[ ]:
```
# Load the Wine Quality dataset from local storage path
df = pd.read_csv('/content/winequality-red.csv', sep=';')

# Display metadata and statistical descriptions
print("First 5 records:")
display(df.head())

print("\nDataset Information:")
df.info()

print("\nStatistical Summary:")
display(df.describe())

print(f"\nDataset Dimensions: {df.shape}")

# Check for missing values
print("\nMissing values per column:")
print(df.isnull().sum())
```

```
First 5 records:
   fixed acidity  volatile acidity  citric acid  residual sugar  chlorides  \
0            7.4              0.70         0.00             1.9      0.076
1            7.8              0.88         0.00             2.6      0.098
```

```
2            7.8              0.76         0.04             2.3      0.092
3           11.2              0.28         0.56             1.9      0.075
4            7.4              0.70         0.00             1.9      0.076

   free sulfur dioxide  total sulfur dioxide  density    pH  sulphates  \
0                 11.0                  34.0   0.9978  3.51       0.56
1                 25.0                  67.0   0.9968  3.20       0.68
2                 15.0                  54.0   0.9970  3.26       0.65
3                 17.0                  60.0   0.9980  3.16       0.58
4                 11.0                  34.0   0.9978  3.51       0.56

   alcohol  quality
0      9.4        5
1      9.8        5
2      9.8        5
3      9.8        6
4      9.4        5


Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1599 entries, 0 to 1598
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   fixed acidity         1599 non-null   float64
 1   volatile acidity      1599 non-null   float64
 2   citric acid           1599 non-null   float64
 3   residual sugar        1599 non-null   float64
 4   chlorides             1599 non-null   float64
 5   free sulfur dioxide   1599 non-null   float64
 6   total sulfur dioxide  1599 non-null   float64
 7   density               1599 non-null   float64
 8   pH                    1599 non-null   float64
 9   sulphates             1599 non-null   float64
 10  alcohol               1599 non-null   float64
 11  quality               1599 non-null   int64
dtypes: float64(11), int64(1)
memory usage: 150.0 KB

Statistical Summary:

       fixed acidity  volatile acidity  citric acid  residual sugar  \
count    1599.000000       1599.000000  1599.000000     1599.000000
mean        8.319637          0.527821     0.270976        2.538806
std         1.741096          0.179060     0.194801        1.409928
min         4.600000          0.120000     0.000000        0.900000
25%         7.100000          0.390000     0.090000        1.900000
50%         7.900000          0.520000     0.260000        2.200000
```

```
75%          9.200000          0.640000     0.420000        2.600000
max         15.900000          1.580000     1.000000       15.500000

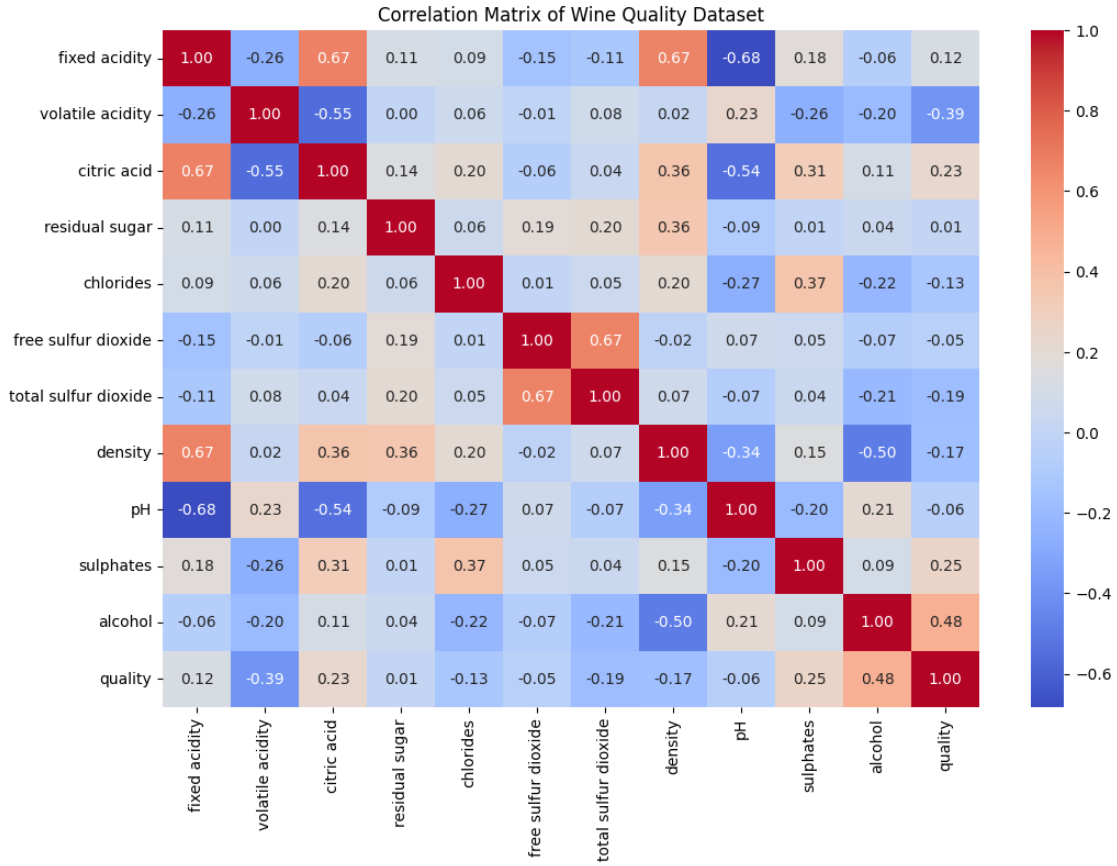
         chlorides  free sulfur dioxide  total sulfur dioxide      density  \
count  1599.000000          1599.000000           1599.000000  1599.000000
mean      0.087467            15.874922             46.467792     0.996747
std       0.047065            10.460157             32.895324     0.001887
min       0.012000             1.000000              6.000000     0.990070
25%       0.070000             7.000000             22.000000     0.995600
50%       0.079000            14.000000             38.000000     0.996750
75%       0.090000            21.000000             62.000000     0.997835
max       0.611000            72.000000            289.000000     1.003690

                pH    sulphates      alcohol      quality
count  1599.000000  1599.000000  1599.000000  1599.000000
mean      3.311113     0.658149    10.422983     5.636023
std       0.154386     0.169507     1.065668     0.807569
min       2.740000     0.330000     8.400000     3.000000
25%       3.210000     0.550000     9.500000     5.000000
50%       3.310000     0.620000    10.200000     6.000000
75%       3.400000     0.730000    11.100000     6.000000
max       4.010000     2.000000    14.900000     8.000000


Dataset Dimensions: (1599, 12)

Missing values per column:
fixed acidity           0
volatile acidity        0
citric acid             0
residual sugar          0
chlorides               0
free sulfur dioxide     0
total sulfur dioxide    0
density                 0
pH                      0
sulphates               0
alcohol                 0
quality                 0
dtype: int64
```

[ ]:
```
plt.figure(figsize=(12, 8))
correlation_matrix = df.corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Matrix of Wine Quality Dataset')
plt.show()
```



[ ]:
```
# Separate Features (X) and Target Variable (y)
X = df.drop(columns=['quality'])
y = df['quality']

print(f"X dimensions: {X.shape}")
print(f"y dimensions: {y.shape}")

# Split the dataset (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
↪random_state=42)

# Normalize the dataset using StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Data preprocessed and scaled successfully.")
```

```
X dimensions: (1599, 11)
y dimensions: (1599,)
```

```
Data preprocessed and scaled successfully.
```

[ ]:
```python
# Define and train the first MLP Architecture
mlp1 = MLPClassifier(hidden_layer_sizes=(50,), max_iter=500, random_state=42)
mlp1.fit(X_train_scaled, y_train)

# Predict and evaluate
y_pred1 = mlp1.predict(X_test_scaled)
accuracy1 = accuracy_score(y_test, y_pred1)

print(f"Architecture 1 (50,) Accuracy: {accuracy1:.4f}")
print("\nClassification Report for Architecture 1:")
print(classification_report(y_test, y_pred1, zero_division=0))
```

```
Architecture 1 (50,) Accuracy: 0.5969

Classification Report for Architecture 1:
              precision    recall  f1-score   support

           3       0.00      0.00      0.00         1
           4       0.00      0.00      0.00        10
           5       0.63      0.72      0.67       130
           6       0.59      0.60      0.60       132
           7       0.55      0.43      0.48        42
           8       0.00      0.00      0.00         5

    accuracy                           0.60       320
   macro avg       0.29      0.29      0.29       320
weighted avg       0.57      0.60      0.58       320


/usr/local/lib/python3.13/dist-
packages/sklearn/neural_network/_multilayer_perceptron.py:691:
ConvergenceWarning: Stochastic Optimizer: Maximum iterations (500) reached and
the optimization hasn't converged yet.
  warnings.warn(
```

[ ]:
```python
# Define and train the second MLP Architecture
mlp2 = MLPClassifier(hidden_layer_sizes=(100, 50), max_iter=500,␣
 ↪random_state=42)
mlp2.fit(X_train_scaled, y_train)

# Predict and evaluate
y_pred2 = mlp2.predict(X_test_scaled)
accuracy2 = accuracy_score(y_test, y_pred2)

print(f"Architecture 2 (100, 50) Accuracy: {accuracy2:.4f}")
print("\nClassification Report for Architecture 2:")
```

```
print(classification_report(y_test, y_pred2, zero_division=0))
```

```
Architecture 2 (100, 50) Accuracy: 0.6406

Classification Report for Architecture 2:
              precision    recall  f1-score   support

           3       0.00      0.00      0.00         1
           4       0.00      0.00      0.00        10
           5       0.69      0.73      0.71       130
           6       0.63      0.65      0.64       132
           7       0.57      0.57      0.57        42
           8       0.00      0.00      0.00         5

    accuracy                           0.64       320
   macro avg       0.32      0.33      0.32       320
weighted avg       0.62      0.64      0.63       320


/usr/local/lib/python3.13/dist-
packages/sklearn/neural_network/_multilayer_perceptron.py:691:
ConvergenceWarning: Stochastic Optimizer: Maximum iterations (500) reached and
the optimization hasn't converged yet.
  warnings.warn(
```

[ ]:
```
# Define and train the third MLP Architecture
mlp3 = MLPClassifier(hidden_layer_sizes=(128, 64, 32), max_iter=500,␣
 ↪random_state=42)
mlp3.fit(X_train_scaled, y_train)

# Predict and evaluate
y_pred3 = mlp3.predict(X_test_scaled)
accuracy3 = accuracy_score(y_test, y_pred3)

print(f"Architecture 3 (128, 64, 32) Accuracy: {accuracy3:.4f}")
print("\nClassification Report for Architecture 3:")
print(classification_report(y_test, y_pred3, zero_division=0))
```

```
Architecture 3 (128, 64, 32) Accuracy: 0.6438

Classification Report for Architecture 3:
              precision    recall  f1-score   support

           3       0.00      0.00      0.00         1
           4       0.43      0.30      0.35        10
           5       0.69      0.74      0.71       130
           6       0.63      0.61      0.62       132
           7       0.60      0.64      0.62        42
           8       0.00      0.00      0.00         5
```

accuracy			0.64	320
macro avg	0.39	0.38	0.38	320
weighted avg	0.63	0.64	0.64	320